Project Name:

To-Do Manager App

Course name:

Programming in Python

Section: **B**

Semester: Spring 2025-2026

Course Teacher:

Md. Tanzeem Rahat

Group No.: **07**

Submitted by:

No	Name	ID	Program
1	Muhammad Talha Islam	22-49599-3	BSc [CSE]
2	Wafa	22-49542-3	BSc [CSE]

Submission Date: 28 -03 -2026

Project Details

Overview

The To-Do Manager App with Priority is a desktop-based task management application developed using Python and the customtkinter GUI library. The application allows users to efficiently organize, track, and manage daily tasks with priority levels and due dates.

This project is designed using proper Object-Oriented Programming (OOP) principles and follows a modular architecture where each component (model, manager, storage, and UI controller) is separated into different files for maintainability and scalability.

The system ensures data persistence using a JSON file, meaning all tasks remain saved even after restarting the application.

Objective

The main objective of this project is to:

- To develop a structured desktop application using Python.
- To implement CRUD operations with proper validation.
- To apply OOP concepts in a real-world system.
- To store and retrieve data using JSON.
- To provide task tracking with summary reporting.

Features

Core Features

- Add new tasks with title, priority, and due date.
- Delete existing tasks.
- Mark tasks as completed.
- View a complete list of tasks in the dashboard.

Search & Sort Functionality

- Search tasks by title (case-insensitive).
- Sort tasks by priority (High → Medium → Low).

Reports

- Total number of tasks
- Number of completed tasks
- Number of pending tasks

Data Persistence

- Tasks are saved in data.json
- Data remains after restarting the app

Validation

- Empty task titles are not allowed.
- Due date must follow the format: YYYY-MM-DD.
- Invalid inputs do not crash the program.
- The application handles errors gracefully.

Project Structure

```
todo_manager/
│
├── main.py            # Entry point
├── ui.py              # GUI (Controller)
├── manager.py         # Business logic
├── models.py          # Task entity
├── storage.py         # JSON storage handler
├── data.json          # Data file (auto-created)
└── requirements.txt   # Dependencies
```

Class Responsibilities

- **Task (models.py)**
  Represents the task entity and defines task attributes.
- **TaskManager (manager.py)**
  Handles all business logic including CRUD operations, search, sorting, and report generation.
- **JsonStorage (storage.py)**
  Manages saving and loading tasks from the JSON file.
- **App (ui.py)**
  Controls the graphical user interface and user interactions.
- **main.py**
  Entry point that launches the application.

Technology Used

- Python 3.x
- customtkinter (External GUI library)
- JSON for data storage
- Virtual Environment (.venv)

Screenshots of To-Do Manager

1. Dashboard (Task List)

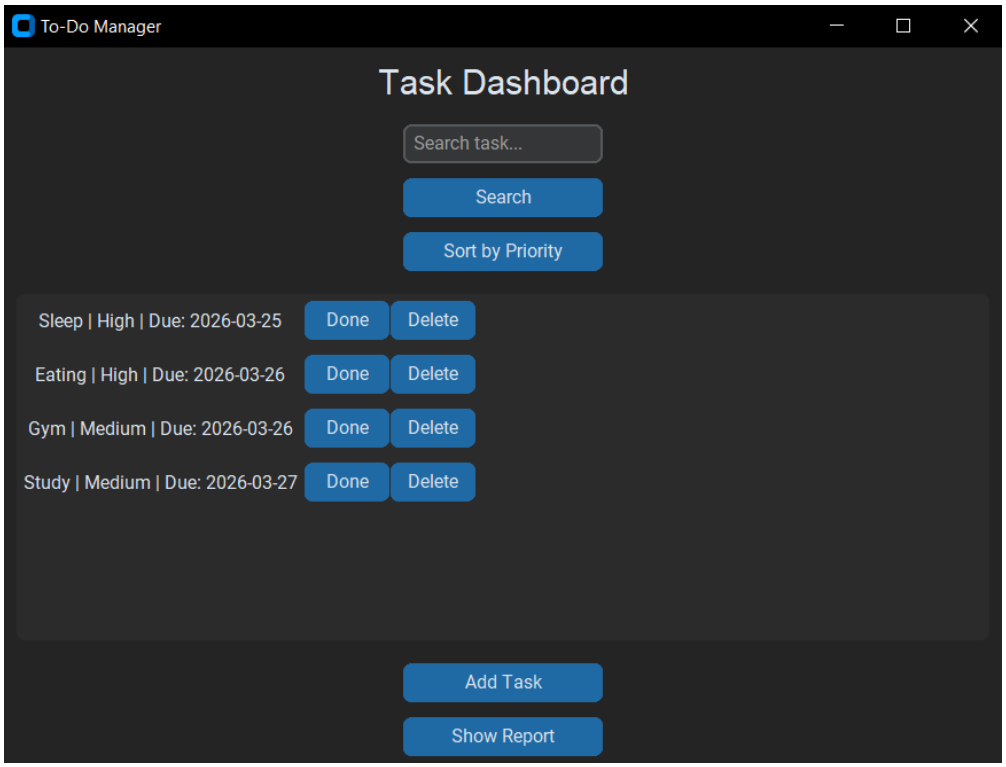

Fig 1: Dashboard View

2. Add Task Window

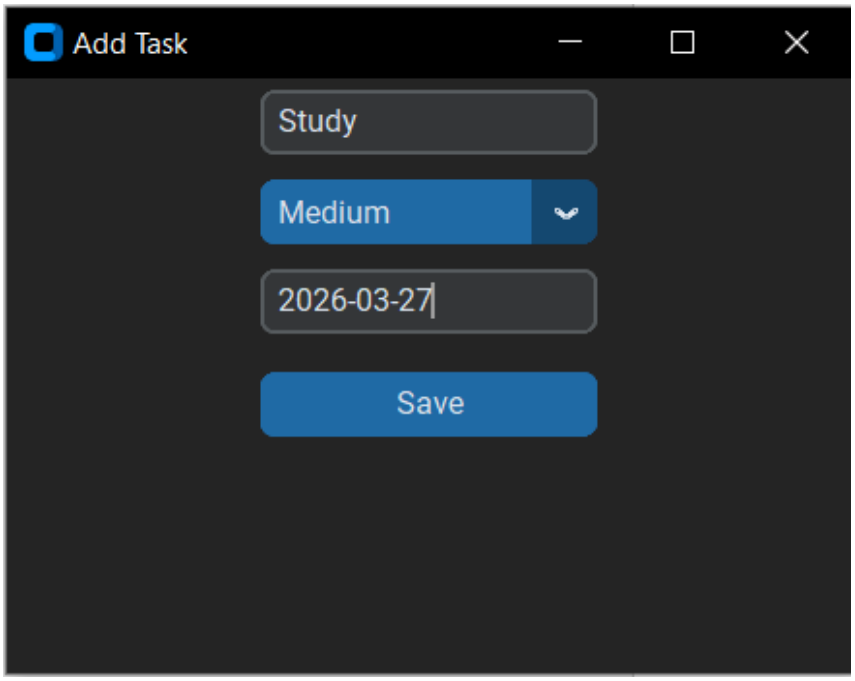

Fig 2: Add Task Window View

3. Search Result

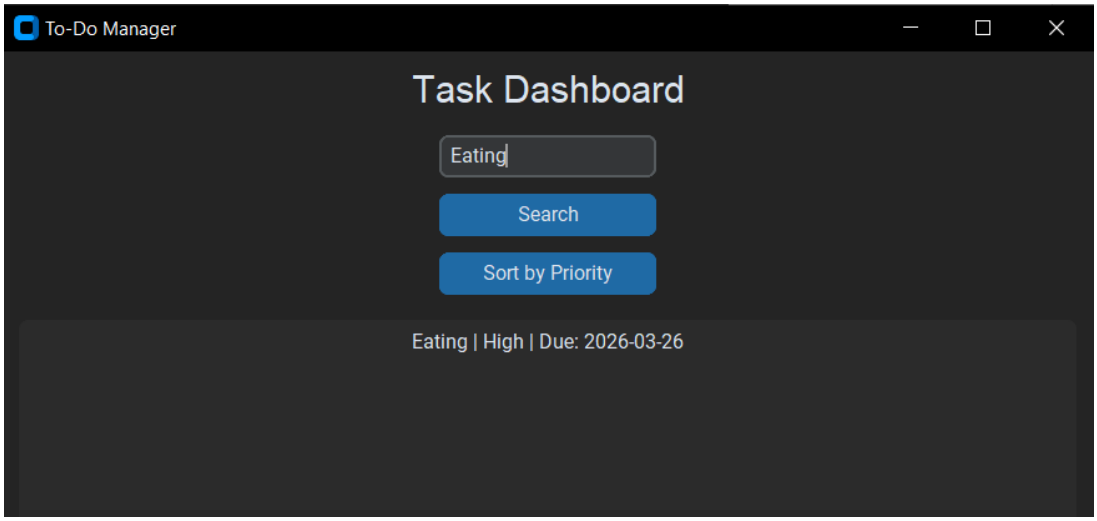

Fig 3: Search Result View

4. Sort Window

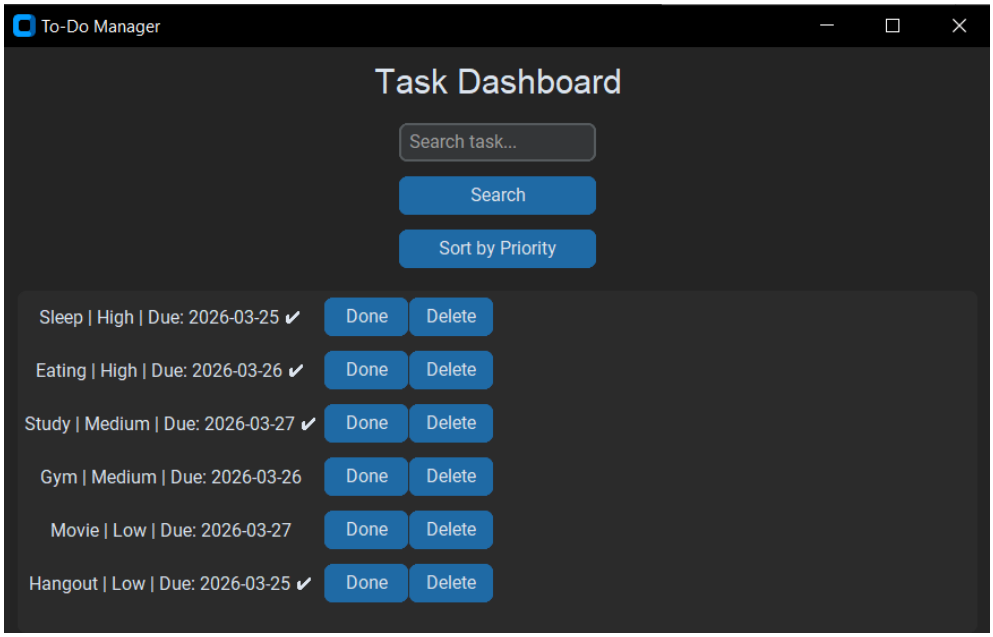

Fig 4: Sort Window View

5. Report Window

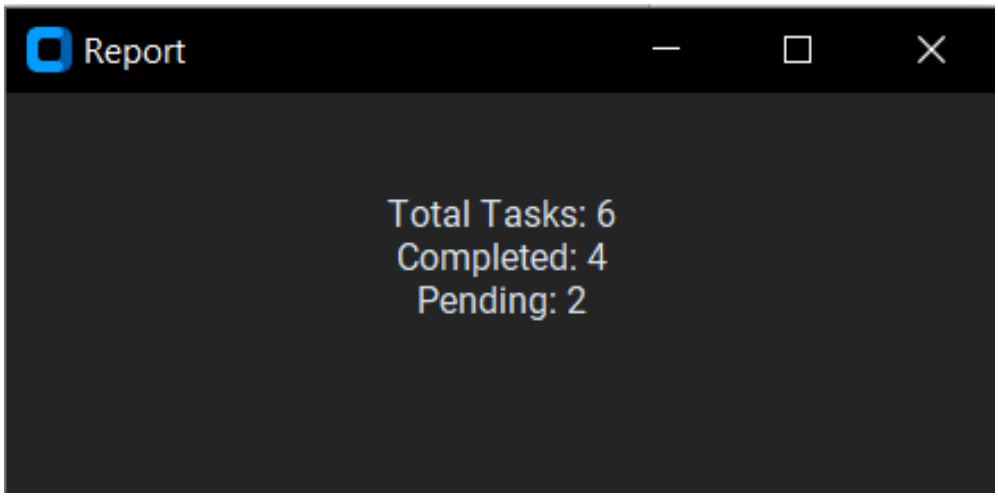

Fig 5: Report Window View

6. Completed Task View

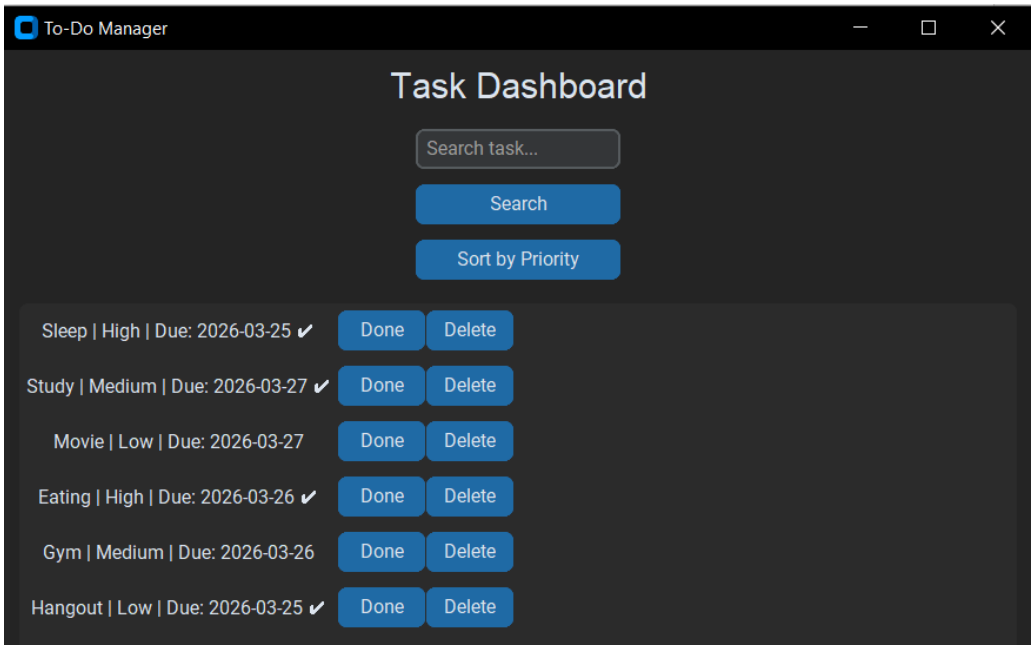

Fig 6: Completed Task View